LOW-LEVEL DESIGN (LLD)

RAG-Based Customer Support Assistant

Module Design | Data Structures | Workflow Logic | API Design | Error Handling

Project	RAG-Based Customer Support Assistant
Version	1.0
Date	May 2026
Files	ingest.py, retrieval.py, generator.py, hitl.py, graph.py, main.py

1. Module-Level Design

ingest.py — Document Processing + Chunking + Embedding + Storage

- Function: ingest_pdfs(pdf_list: List[str]) -> None
- Iterates over each PDF path in pdf_list
- PyPDFLoader().load() returns List[Document] with .page_content and .metadata
- Attaches source path to each doc: doc.metadata['source'] = pdf_path
- RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)
- HuggingFaceEmbeddings(model_name='all-MiniLM-L6-v2') — local, free
- Chroma(persist_directory='db', embedding_function=embeddings)
- vectorstore.add_documents(chunks) + vectorstore.persist()
- Error handled via try/except — prints error and exits gracefully

retrieval.py — Vector Search + Context Extraction

- Global objects: embeddings, vectorstore (initialized at module load)
- Function: retrieve_docs(query: str) -> Optional[str]
- Normalizes query: query.strip().lower()
- vectorstore.similarity_search_with_score(query, k=3) returns List[Tuple[Document, float]]
- Score threshold: if best_score > 1.5 → return None (triggers escalation)
- Filters lines: skips empty lines and lines starting with digit (headings)
- Keyword match: checks if any query word appears in a line
- Returns best_line.capitalize() or first non-heading line as fallback
- Returns None on any exception (safe failure)

generator.py — LLM Answer Generation

- client = OpenAI(api_key=os.getenv('OPENAI_API_KEY'))
- Function: generate_answer(query: str, context: str) -> Optional[str]
- Builds structured prompt with Context + Question sections

- API call: client.chat.completions.create(model='gpt-4o-mini', temperature=0.3)
- Returns response.choices[0].message.content
- Returns None on LLM failure — triggers HITL in graph

hitl.py — Human-in-the-Loop Escalation

- Function: human_fallback(query: str) -> str
- Prints escalation warning + the original user query
- Calls input() to get a human-typed response from terminal
- Returns the human response string to be used as final answer

graph.py — LangGraph Workflow Engine

- process_node(state: dict) -> dict:
- Calls state['retriever'](query) → gets context or None
- If context is None → state['escalate'] = True, return state
- Sets state['answer'] = context, state['escalate'] = False
- output_node(state: dict) -> dict:
- If state['escalate'] == True → calls state['hitl'](query)
- Otherwise returns state with existing answer
- build_graph() → StateGraph(dict) with nodes + edge + compile()

main.py — Entry Point + Orchestration

- Checks if ./db exists → skips ingestion if already done
- Calls ingest_pdfs([list of pdf paths]) on first run
- Calls build_graph() to get compiled LangGraph app
- Chat loop: while True → input() → build state dict → graph.invoke(state)
- State dict keys: query, retriever, generator, hitl, answer, escalate
- Prints result['answer'] after each invocation

2. Data Structures

Document Representation (LangChain Document object):

```
Document(
page_content: str, # raw text of the page/chunk
metadata: dict # {'source': 'path/to/file.pdf', 'page': 0}
)
```

Chunk Format (after text splitting):

```
List[Document] # each chunk is a Document with ~500 chars
# chunk_size=500, chunk_overlap=100
# total chunks = depends on PDF size
```

Embedding Structure (ChromaDB internal):

```
# HuggingFace all-MiniLM-L6-v2 produces:
embedding: List[float] # 384-dimensional vector
# Stored in ChromaDB with associated document text + metadata
```

Query-Response Schema:

```
# similarity_search_with_score returns:
List[Tuple[Document, float]]
# float = cosine distance score (lower = more similar)
# threshold: score > 1.5 → irrelevant → escalate
```

State Object for LangGraph:

```
state = {
'query': str, # user's question
'retriever': Callable, # retrieve_docs function
'generator': Callable, # generate_answer function
'hitl': Callable, # human_fallback function
'answer': str, # final answer (filled by nodes)
'escalate': bool, # True = route to human
}
```

3. Workflow Design (LangGraph)

Element	Name	Responsibility
Node 1	process_node	Retrieve context from ChromaDB. Set escalate flag based on result.
Node 2	output_node	Check escalate flag. Call HITL if True; else return stored answer.
Edge	process → output	Single unconditional edge. Routing logic is inside output_node.
State	dict (TypedDict-compatible)	Carries all data between nodes: query, answer, escalate, callables.
Entry	set_entry_point('process')	Graph execution always starts at process_node.

4. Conditional Routing Logic

Answer Generation Criteria:

context is not None AND context string is not empty AND score <= 1.5

Escalation: No chunks found:
vectorstore returns empty list → retrieve_docs() returns None → escalate=True

Escalation: Low confidence:
best_score > 1.5 (cosine distance too high) → context not relevant → escalate=True

Escalation: Missing context:
All lines are headings or empty → no meaningful content returned → escalate=True

Escalation: LLM failure:
generate_answer() returns None → output_node detects and escalates

5. HITL Design

When escalation is triggered: state['escalate'] == True inside output_node.

What happens after escalation: human_fallback(query) is called. It prints the unresolved query and waits for a human operator to type a response via terminal.

How human response is integrated: The typed string is returned and stored in state['answer'], then printed to the user as the final answer — same output format as AI answers.

6. API / Interface Design

Input Format: Plain text string via terminal input(). No preprocessing required by user.

Output Format: Plain text string printed to terminal. Prefix: '■ Final Answer:'

Interaction Flow: Synchronous, blocking CLI loop. Each query-response cycle is atomic. Type 'exit' to quit.

7. Error Handling

Error Scenario	Handling Strategy
Missing data / PDF not found	ingest.py try/except catches FileNotFoundError. Prints error and continues with next PDF.
No relevant chunks found	retrieve_docs() returns None. process_node sets escalate=True. HITL takes over.
LLM API failure	generate_answer() returns None. output_node detects None answer and escalates to HITL.
ChromaDB connection error	retrieval.py try/except returns None. Escalation is triggered automatically.